Getting started with Repository pattern in ASP.NET Core Web App

The Repository pattern is a design pattern that abstracts away the data access logic from the rest of the application. It acts as an intermediary between the domain logic and the data source, providing a set of standardized methods for retrieving and storing data. This decoupling makes the application more maintainable, testable, and adaptable.

Definition: The Repository pattern involves defining a repository interface that declares the operations allowed on the domain model. A concrete implementation of the repository interface handles the actual data access using a specific technology (e.g., Entity Framework Core).

Benefits:

- Maintenance: Simplifies maintenance by centralizing data access logic in one place.
- Testability: Facilitates unit testing by allowing developers to mock the repository implementation.
- Adaptability: Enables easy adaptation to different data sources or technologies without impacting the core logic.
- Cleaner Code: Separates concerns, leading to cleaner and more organized code.

Limitations:

- Complexity: Can add initial complexity to small or simple projects where direct data access is sufficient.
- Overhead: May introduce slight performance overhead due to the abstraction layer.

Best Practices:

- Generic Repositories: Consider using generic repositories for common CRUD operations.
- Avoid Exposing EF Core Specifics: Ensure that repository interfaces are generic and not specific to any particular technology.
- Repository per Entity: In general, design a separate repository for each major entity in your domain model.

Getting started with AJAX

AJAX (Asynchronous JavaScript and XML) is a technique that enables web pages to update content dynamically without reloading the entire page. This allows for a more smooth and engaging user experience.

Definition: AJAX involves using JavaScript to send asynchronous requests to a server-side script and receive data in response (often XML, JSON, or HTML). The received data is then used to update specific portions of the web page.

Benefits:

- Better User Experience: Dynamic content updates lead to a more interactive and smoother user experience.
- Faster Updates: Smaller data transfers and fewer page reloads result in faster content updates.
- Partial Page Reloads: Avoids full page refreshes, improving perceived performance.

Limitations:

- Complexity: Requires knowledge of JavaScript and asynchronous programming.
- SEO Issues: Content loaded dynamically via AJAX may not be easily crawlable by search engines.
- Browser Compatibility: Some AJAX features might behave differently in older browsers.

Best Practices:

- Graceful Degradation: Ensure that the web page is still functional and usable even if JavaScript or AJAX is disabled.
- Loading Indicators: Provide visual feedback (e.g., loading spinner) while AJAX requests are in progress.
- Server-side Validation: Even with AJAX, maintain robust server-side validation for security.

Asp.Net Core Advanced

Asp.Net Core offers several advanced features and techniques for building high-performance and scalable web applications. Some of the common advanced topics include:

Input Validation

Input validation is a crucial step in ensuring data integrity and security. It involves checking user input against a set of rules to make sure it meets specific criteria.

Why do we need validations?

- Data Integrity: Prevent invalid or malicious data from being stored in the database.
- User Experience: Provide feedback to users when they enter invalid data, helping them correct mistakes.
- Security: Mitigate common attacks like SQL injection and cross-site scripting (XSS) by validating input.

Types of validations:

- Client-side Validation: Performed in the browser using JavaScript for immediate feedback.
- Server-side Validation: Executed on the server for security and consistency, even if client-side is bypassed.

Output Encoding

Output encoding is the process of converting data into a form that can be safely rendered in a web browser, preventing attacks like Cross-Site Scripting (XSS).

What is Encoding in ASP.NET Core Web App? Encoding in ASP.NET Core involves converting special characters (e.g., `<`, `>`, `&`) into their equivalent HTML entities (e.g., `<` becomes `<`, `>` becomes `>`). This ensures that the browser interprets them as literal characters rather than HTML tags.

Why do we need that, practical application of it?

- Prevent XSS: Output encoding is the primary defense against XSS attacks, where malicious scripts are injected into web pages.
- Practical Application: Use `@Html.Encode()` or other built-in encoding functions in your views to automatically encode dynamic content before displaying it.

Authentication and Authorization

Authentication and authorization are the cornerstones of securing access to your application's resources. Authentication involves verifying the identity of a user, while authorization determines what actions they are allowed to perform.

Use cases:

- Restricting access to certain pages or actions based on user roles (e.g., admin only).
- Personalizing content and features based on the logged-in user.
- Tracking user activity and audit trails.

Types:

- Authentication: Cookie-based, JWT-based, OAuth/OpenID Connect.

- Authorization: Role-based, Policy-based.

Applications:

- Secure access to sensitive data and critical operations.
- Create multi-user applications with granular access controls.

Securing MVC Application

Securing an MVC application involves implementing various measures at different levels to protect against common web vulnerabilities and unauthorized access.

Different ways and steps:

1. **Authentication and Authorization:** Implement robust authentication and authorization mechanisms as described above.
2. **Input Validation:** Employ server-side validation to sanitize user input and prevent attacks.
3. **Output Encoding:** Encode all user-generated content before rendering it in the browser to prevent XSS.
4. **Prevent Cross-Site Request Forgery (CSRF):** Use anti-CSRF tokens to ensure that requests originate from authorized users.
5. **HTTPS:** Enforce the use of HTTPS for all communication to protect data in transit.
6. **Secure Configuration Management:** Avoid storing sensitive information (e.g., connection strings, API keys) in plain text.
7. **Cross-Origin Resource Sharing (CORS):** Configure CORS policy properly to restrict access to your APIs from unauthorized domains.

JWT Token and OAuth

JWT (JSON Web Token) is an open standard for securely transmitting information between parties as a JSON object. OAuth is an open protocol that allows users to grant third-party applications limited access to their resources on another server, without sharing their credentials.

- **JWT:** Often used for stateless authentication in modern web applications and APIs. It contains user information and claims signed by a trusted authority.
- **OAuth:** Enables users to log in to your application using their existing credentials from providers like Google, Facebook, etc.

Database Security

Database security is essential to protect your application's data from unauthorized access, modifications, and

theft.

Security Practices:

- **Principle of Least Privilege:** Grant users and applications only the minimum necessary privileges required to perform their tasks.
- **SQL Injection Prevention:** Use parameterized queries or ORMs like EF Core to prevent SQL injection attacks.
- **Encryption at Rest:** Encrypt sensitive data stored in the database.
- **Regular Backups:** Implement a regular backup and recovery plan to minimize data loss in case of an incident.
- **Strong Passwords:** Use strong and unique passwords for database users and administrators.

Security Practices in SDLC

Security should be an integral part of the entire Software Development Life Cycle (SDLC) to proactively identify and address security risks throughout the development process.

Best Practices:

- **Security Requirements Gathering:** Define security requirements early in the project.
- **Security Architecture and Design Review:** Review design and architecture for potential security flaws.
- **Secure Coding Guidelines:** Establish and follow secure coding standards and guidelines.
- **Security Testing:** Conduct regular security testing, including static analysis, dynamic analysis, and penetration testing.
- **Security Training:** Provide regular security training to development teams.
- **Incident Response Planning:** Develop a plan to respond effectively to security incidents.

By incorporating these detailed topics and security practices into your ASP.NET Core development process, you can build more secure, reliable, and trustworthy web applications.

Getting started with Repository pattern in ASP.NET core Web App:

Problem Statement

A training institute wants to build a Student Management System using ASP.NET Core MVC and Entity Framework Core.

The application should allow users to:

- Add new students
- View student details
- Edit student information
- Delete students
- Store data in SQL Server

The management also wants:

- Clean separation between Business Logic and Data Access Logic
- Easy testing and maintenance
- Reusable database operations

To achieve this, we will implement the Repository Pattern in an ASP.NET Core Web App.

What is Repository Pattern?

The Repository Pattern acts as a middle layer between:

- Controller (Business Logic)
- Database/Data Access Layer

It hides database operations and provides methods like:

- GetAll()
- GetById()
- Add()
- Update()

- Delete()

Controller

↓

Repository Interface (New Layer (unit of work))

↓

Repository Class

↓

Entity Framework Core DbContext

↓

SQL Server Database

Step 1: Create ASP.NET Core MVC Project

Step 2: Install Required Packages

Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools

Step 3: Create Model Class ie Student.cs

```
public class Student
{
[Key]
public int StudentId { get; set; }
[Required]
public string Name { get; set; }
public string Course { get; set; }
public int Age { get; set; }
}
```

Step 4: Create DbContext Class

(after creating Data folder we will create ApplicationDbContext.cs)

```
public class ApplicationDbContext : DbContext
```

```
{

public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)

: base(options)

{

}



public DbSet<Student> Students { get; set; }

}
```

Step 5: Configure Connection String

```
{

  "ConnectionStrings": {

    "DefaultConnection":
"Server=.;Database=StudentDB;Trusted_Connection=True;TrustServerCertificate=True"

  }

}
```

Step 6: Register DbContext in Program.cs

```
using Microsoft.EntityFrameworkCore;

using StudentRepositoryDemo.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(

builder.Configuration.GetConnectionString("DefaultConnection")));
```

Step 7: Create Repository Interface

```
public interface IStudentRepository
   {
      IEnumerable<Student> GetAllStudents();

      Student GetStudentById(int id);

      void AddStudent(Student student);

      void UpdateStudent(Student student);

      void DeleteStudent(int id);

      void Save();
   }
```

Create a folder repository with
IStudentRepository.cs

Step 8: Create Repository Class StudentRepository.cs

```
public class StudentRepository : IStudentRepository
{
private readonly ApplicationDbContext _context;

public StudentRepository(ApplicationDbContext context)
{
_context = context;
}

public IEnumerable<Student> GetAllStudents()
{
return _context.Students.ToList();
}

public Student GetStudentById(int id)
{
return _context.Students.Find(id);
}

public void AddStudent(Student student)
{
_context.Students.Add(student);
}

public void UpdateStudent(Student student)
{
_context.Students.Update(student);
}

public void DeleteStudent(int id)
{
var student = _context.Students.Find(id);

if (student != null)
{
_context.Students.Remove(student);
}
}

public void Save()
{
```

```
_context.SaveChanges();
}
}
```

Step 9: Register Repository in Dependency Injection

in Program.cs we will do following changes

```
using StudentRepositoryDemo.Repository;

builder.Services.AddScoped<IStudentRepository, StudentRepository>();
```

Step 10: Create Controller

StudentController.cs

```
public class StudentController : Controller
{
private readonly IStudentRepository _repository;

public StudentController(IStudentRepository repository)
{
_repository = repository;
}

public IActionResult Index()
{
var students = _repository.GetAllStudents();

return View(students);
}

public IActionResult Create()
{
return View();
}

[HttpPost]
public IActionResult Create(Student student)
{
if (ModelState.IsValid)
```

```
{
_repository.AddStudent(student);
_repository.Save();

return RedirectToAction("Index");
}

return View(student);
}
}
```

Step 11: Create Migration

- Add-Migration InitialCreate
- Update-Database

Step 12: Create Razor Views

Views/Student

```
@model IEnumerable<StudentRepositoryDemo.Models.Student>

<h2>Student List</h2>

<a asp-action="Create">Add Student</a>

<table class="table">
   <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Course</th>
      <th>Age</th>
   </tr>

   @foreach (var item in Model)
   {
      <tr>
         <td>@item.StudentId</td>
         <td>@item.Name</td>
         <td>@item.Course</td>
         <td>@item.Age</td>
```

```
        </tr>
    }
</table>
```

StudentRepositoryDemo

```
│
├── Controllers
│   └── StudentController.cs
│
├── Models
│   └── Student.cs
│
├── Data
│   └── ApplicationDbContext.cs
│
├── Repository
│   ├── IStudentRepository.cs
│   └── StudentRepository.cs
│
├── Views
│   └── Student
│       ├── Index.cshtml
│       └── Create.cshtml
```

Step No.	Task	File/Folder	Action
1	Create ASP.NET Core MVC Project	Visual Studio	Create a new ASP.NET Core MVC Web App project named StudentRepositoryDemo
2	Install EF Core Packages	NuGet Package Manager	Install Microsoft.EntityFrameworkCore.SqlServer and Microsoft.EntityFrameworkCore.Tools
3	Create Model Class	Models/Student.cs	Create Student entity with StudentId, Name, Course, and Age properties
4	Create DbContext Class	Data/ApplicationDbContext.cs	Configure DbContext and DbSet for Student table
5	Configure Connection String	appsettings.json	Add SQL Server database connection string
6	Register DbContext	Program.cs	Configure Entity Framework Core with SQL Server

7	Create Repository Interface	Repository/IStudentRepository.cs	Define CRUD method declarations
8	Create Repository Class	Repository/StudentRepository.cs	Implement CRUD operations using DbContext
9	Register Repository Service	Program.cs	Add repository dependency injection configuration
10	Create Controller	Controllers/StudentController.cs	Implement actions for displaying and adding students
11	Create Migration	Package Manager Console	Run migration command to generate database schema
12	Update Database	Package Manager Console	Execute database update command
13	Create Views Folder	Views/Student	Create Razor View folder for Student module
14	Create Index View	Views/Student/Index.cshtml	Display list of students in table format
15	Create Create	Views/Student/Create.cshtml	Create form for adding student details

	View		
16	Run Application	Browser/Application	Execute project and test CRUD functionality
17	Verify Database Tables	SQL Server	Check Student table creation and inserted records
18	Test Repository Operations	Application UI	Validate Add, View, Update, and Delete operations
19	Implement Dependency Injection Flow	Application Architecture	Verify Controller → Repository → DbContext workflow
20	Extend Application	Project Enhancement	Add Edit, Delete, Search, Async, and Generic Repository features

The Problem Statement & Scenario

We need to create a simple page where a user can enter a Product ID and click a button. The application must then retrieve the Product Price from the server and display only the price *on the current page*, without triggering a full page reload.

Technology Stack:

- ASP.NET Core MVC (using .NET 8.0/7.0/6.0)
- jQuery (standard $.ajax method)

---

2. Implementation Steps

We will proceed in six clear stages, starting from the Model and moving to the Client script.

Prerequisites

We assume you have a basic, running ASP.NET Core MVC project created from the standard template.

Phase 1: Define the Data Model

Since this is a simple demo, we won't use a real database. We'll create a dummy "Entity" and a static list to act as our repository.

Step 1: Create a new folder named Data in your project (optional, but good practice).Step 2: Create a file named Product.cs inside Models or Data folder.

Models/Product.cs

Phase 2: Create the Controller & Repository Logic

We need a controller to serve the main HTML page and another action to handle the asynchronous data request.

Step 1: Create ProductController.cs in the Controllers folder.

Controllers/ProductController.cs

Phase 3: Set up the Initial View (Index.cshtml)

We need the HTML structure where the user interacts and where the partial update will happen. This View is served by the Index() action.

Step 1: In the Views folder, create a new folder named Product.Step 2: Inside Views/Product, create Index.cshtml.

Views/Product/Index.cshtml

Phase 4: Write the jQuery AJAX Script

This is where the magic happens. We attach a jQuery script to the Scripts section of the View. It binds to the button click, sends the ID, and handles the JSON response.

Step 1: In Views/Product/Index.cshtml, add the following inside the @section Scripts { ... } block (which was empty in Phase 3).

Views/Product/Index.cshtml

Phase 5: Final Demo & Test Output

1. Page Load: Access /Product/Index. The page appears, but the results alert is hidden.2. Interaction: Enter 1 in the ID field and click Find Price.3.

The AJAX Moment: The button becomes disabled, the spinner spins (0.8s wait simulated), and only the Price appears below. *The entire rest of the page (header, input, button) has remained stationary and untouched.*

Event	ID Input	Alert Area	UI Interaction	State
Initial GET	--	Hidden	Serve main HTML	Full Reload
User Clicks Btn	"2"	Hidden (Old)	*N/A*	Stationary
During Request	"2"	Hidden	Button disabled, Spinner active	Asynchronous
Request Success	"2"	Shows: $799.00	Spinner hidden, Alert visible	Stationary

Request Failed	"99"	Shows: ID not found	Spinner hidden, Error visible	Stationary

---

Key Takeaways for AJAX in MVC

1. Separation of Concerns: One Controller action serves the full View (HttpGet: Index), and *another dedicated action* handles the data exchange (HttpPost: GetPrice returning JsonResult).
2. JsonResult: The data endpoint *must* return JsonResult so that jQuery can easily parse the response as a JavaScript object.
3. No full reload: We are *not* returning a RedirectToActionResult or a ViewResult in Phase 2; we only send a small data payload back.
4. Security Best Practice: Even though jQuery does client-side validation, *always* maintain robust validation in your GetPrice controller action. Malicious users can bypass JavaScript entirely.

Step-by-Step: AJAX Implementation in ASP.NET Core MVC

Phase / Step	Action	File Location	Key Concepts & Description
Phase 1: Data Model	Create the Entity	Models/Product.cs	Define a simple Product class with properties like Id, Name, and Price to represent the data structure.
Phase 2: Controller	Setup UI Endpoint	Controllers/ProductController.cs	Create an [HttpGet] Index() action. This simply returns the View() to serve the initial, blank HTML page to the user.

	Setup AJAX Endpoint	Controllers/ProductController.cs	Create an [HttpPost] GetPrice(int id) action. This simulates a database lookup and must return a JsonResult (e.g., return Json(new { success = true, price = ... });).
Phase 3: View (UI)	Build the Interface	Views/Product/Index.cshtml	Create the HTML layout containing an input field (#productId), a trigger button (#btnUpdatePrice), and hidden div elements (#resultArea, #errorArea) to display the incoming data.
	Setup Script Section	Views/Product/Index.cshtml	Include the @section Scripts { } block at the bottom of the view. This ensures the JavaScript loads after the main layout (including jQuery) is rendered.
Phase 4: AJAX Script	Bind Click Event	Views/Product/Index.cshtml (in Scripts)	Use jQuery ($("#btnUpdatePrice").click(function() {...})) to capture the user's input when the button is clicked.
	Execute AJAX Call	Views/Product/Index.cshtml (in Scripts)	Implement $.ajax({ ... }) targeting the URL /Product/GetPrice. Set the method to POST, pass the ID in the data payload, and set dataType to json.
	Handle Response	Views/Product/Index.cshtml (in Scripts)	Inside the success: function(response) block, manipulate the DOM (e.g., $("#productPrice").text(response.price)) to show the new data. Handle failures in the error callback.

Phase 5: Testing	Verify Implementation	Web Browser	Run the application, navigate to /Product/Index, enter a valid ID (e.g., 1 or 2), click the button, and observe the specific UI elements update asynchronously without a full page reload.